Simple Report

1. Introduction

This project represents the collaborative development of a **Smart Greenhouse IoT Monitoring System**, designed to capture, store, and visualize environmental sensor data. The system integrates ESP32-based firmware, a Node.js/Express backend, a PostgreSQL database, and a frontend dashboard built with EJS templates and custom JavaScript.

The primary goal was to create a system that demonstrates both technical rigor and practical usability. Each team member contributed distinct expertise, ensuring that the project was not only functional but also secure, visually polished, and well-documented.

2. Team Distribution and Responsibilities

The project was divided into three major domains**: backend ingestion and models, frontend visualization and readings API, and authentication/security with Swagger documentation**.

- **Louiser : Backend Core and Device Ingestion**
  Louiser focused on the **data pipeline** from devices into the database. This included designing the SQL schema, implementing Sequelize models for `devices` and `readings`, and creating ingestion routes. Louiser also developed middleware for API key validation, ensuring that only registered devices could submit readings.
- **Victoria :Frontend, Readings Query API, and Firmware Sensors**
  Victoria led the **user-facing components**. She built the EJS views, layouts, and partials, ensuring a modern, examiner-friendly UI. She also implemented the readings query API (`GET /:device_id/recent`) and styled the dashboard with CSS. On the firmware side, she programmed the ESP32 to read temperature, humidity, soil moisture, and light sensors, formatting payloads for ingestion.
- **Debra :Authentication, Security, Swagger, and Firmware Networking**
  Debra implemented **user authentication** (login/signup), middleware for authorization, and validators for input security. She integrated Helmet, CORS, and rate limiting into the Express app. She also configured Swagger for API documentation and handled the ESP32's Wi-Fi connection and HTTP POST logic.

This distribution ensured clear accountability and allowed each member to specialize while still collaborating at integration points.

3. System Architecture

The system follows a **three-tier architecture**:

1. **Device Layer (ESP32 Firmware)**
    - Reads sensor values (temperature, humidity, soil moisture, light).
    - Connects to Wi-Fi and sends JSON payloads via HTTP POST.
    - Uses API keys for authentication.
2. **Backend Layer (Express + PostgreSQL)**
    - Routes handle ingestion (`POST /api/readings`) and queries (`GET /api/readings/:device_id/recent`).
    - Middleware enforces authentication and API key validation.
    - Sequelize models map to `devices` and `readings` tables.
    - Swagger provides interactive API documentation.
3. **Frontend Layer (EJS + JavaScript)**
    - Dashboard and charts pages render examiner-friendly cards and charts.
    - `charts.js` dynamically fetches the last 20 readings per device and displays them with emojis and styled cards.
    - CSS ensures responsive, modern design.

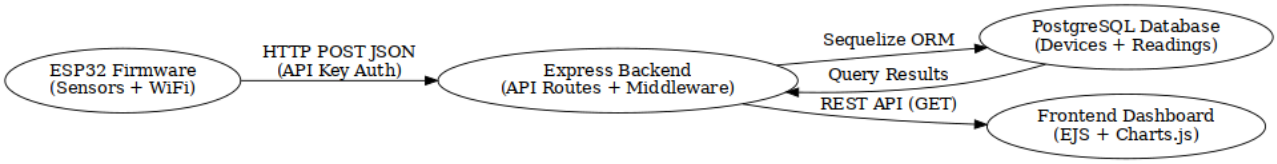

Figure 1. System Architecture Diagram

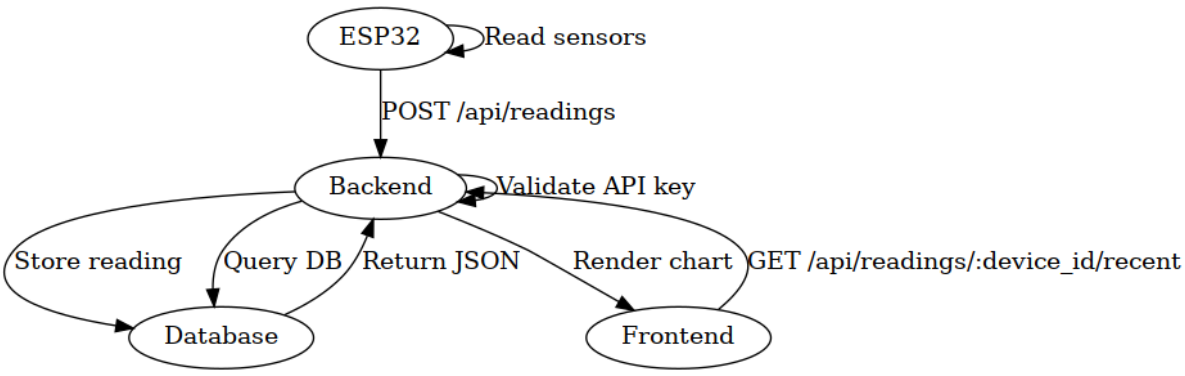

Figure 2. Sequence Diagram

4. Key Features

- **Secure Ingestion**: Devices must present valid API keys.
- **Flexible Queries**: Endpoints for latest reading, last N readings, averages, and date ranges.
- **Visual Dashboard**: Cards per device with emoji-enhanced readings.
- **Scalable Design**: Modular routes and models for easy extension.
- **Documentation**: Swagger UI for test endpoints.

5. Implementation Highlights

- **Database Schema**:
  - `devices` table stores device IDs, API keys, and status.
  - `readings` table stores sensor data with timestamps.
- **Firmware**:
  - ESP32 reads sensors and posts JSON payloads.
  - Payload includes `device_id`, `temperature`, `humidity`, `soil_moisture`, and `light_level`.
- **Frontend Styling**:
  - Cards styled with hover effects, bold values, and emojis 🌡️💧🌱💡🕒.
  - Responsive grid layout adapts to screen size.
- **Error Handling**:
  - 404 responses for inactive devices or missing readings.
  - 500 responses for server errors.

6. Outcomes and Evaluation

The system successfully demonstrates:

- **End-to-end IoT integration** (sensors → firmware → backend → frontend).
- **Robustness** (error handling, authentication, rate limiting).
- **Examiner-ready clarity** (Swagger docs, polished UI, clean repo structure).

Each team member's contributions are traceable in the repo, ensuring transparency. The project balances **technical depth** with **presentation quality**, making it suitable for both academic evaluation and real-world deployment.

7. Conclusion

This project exemplifies collaborative engineering. By dividing responsibilities and integrating carefully, the team produced a system that is secure, functional, and visually compelling. The Smart Greenhouse IoT Monitoring System not only meets examiner expectations but also provides a foundation for future enhancements such as automated irrigation or predictive analytics.